

Содержание

1. Цель работы	2
2. Теоретическое введение	2
2.1. Часть I: Генерация выборок и переобучение	2
2.2. Часть II: Элементарный перцептрон	3
3. Часть I. Генерирование обучающих и тестовых выборок	3
3.1. Общие параметры генерации	4
3.2. Обучающая, валидационная и тестовая выборки	4
3.3. Концентрические окружности (circles)	4
3.4. XOR-распределение	6
3.5. Гауссовские кластеры (blobs)	8
3.6. Спирали Архимеда (spiral)	10
4. Часть II. Реализация элементарного перцептрона	12
4.1. Математическая модель	12
4.2. Код реализации	12
4.3. Где это реализовано в проекте	13
4.4. Вычислительный граф и локальные производные	14
4.5. Обучение и оценка	15
4.5.1. Визуализация результатов блока 2	16
4.5.2. Матрицы ошибок (confusion matrix)	17
4.5.3. Классификационные метрики	17
4.6. Сравнение двух моделей	18
5. Выводы	18
6. Структура проекта	19
7. Репозиторий	19

1. Цель работы

Цель лабораторной работы — исследовать задачи бинарной классификации на двумерных синтетических данных различной структуры и сравнить поведение моделей при линейно и нелинейно разделимых распределениях.

В первой части реализуется генерация обучающих и тестовых выборок

$$\{(x^{(i)}, y^{(i)})\}_{i=1}^N$$

, где $x = (x_1, x_2) \in \mathbb{R}^2$, $y \in \{0, 1\}$, для четырёх типов распределений: circles, XOR, blobs, spiral. Дополнительно моделируется ошибка измерения признаков и анализируется переобучение в `playground.tensorflow.org`.

Во второй части реализуется элементарный перцептрон с двумя функциями активации (ступенчатой и сигмодой), проводится обучение на полученных выборках, строятся матрицы ошибок и сравниваются время обучения и качество классификации.

2. Теоретическое введение

2.1. Часть I: Генерация выборок и переобучение

Рассматриваются четыре типа синтетических распределений в двумерном пространстве признаков:

- концентрические окружности (circles);
- XOR-структура из четырёх кластеров;
- гауссовские кластеры (blobs);
- спирали Архимеда (spiral).

Каждое наблюдение имеет вид:

$$x^{(i)} = (x_1^{(i)}, x_2^{(i)}), \quad y^{(i)} \in \{0, 1\}.$$

Ошибка измерения моделируется добавлением гауссова шума к признакам:

$$x_{\sim} = x + \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, \sigma^2 I_2).$$

Задачи circles, XOR и spiral являются **нелинейно разделимыми**: их разделяющая граница не может быть представлена прямой $w_0 + w_1 x_1 + w_2 x_2 = 0$. Для blobs при достаточном расстоянии между центрами задача является линейно разделимой.

Переобучение фиксируется по разрыву между качеством на train и test:

$$L_{\text{train}} \ll L_{\text{test}},$$

что указывает на запоминание обучающей выборки вместо извлечения обобщающей зависимости.

2.2. Часть II: Элементарный перцептрон

Пусть входной вектор расширен фиктивным признаком:

$$x_{\sim} = (x_0, x_1, x_2)^T, \quad x_0 = 1.$$

Вектор параметров:

$$w_{\sim} = (w_0, w_1, w_2)^T.$$

Прямой проход перцептрона:

$$a = w_{\sim}^T x_{\sim}, \quad y_{\sim} = \varphi(a).$$

Рассматриваются две функции активации:

1) Ступенчатая функция

$$\varphi_{\text{step}(a)} = 1 \quad \text{при } a \geq 0, \text{ иначе } 0.$$

Для неё используется правило обучения перцептрона (теорема о сходимости), т.к. градиентный подход неприменим:

$$\Delta w_{\sim} = \eta \cdot y_{\sim} \cdot x_{\sim}, \quad y_{\sim} = 2y - 1.$$

2) Сигмоида

$$\varphi_{\sigma(a)} = \frac{1}{1 + e^{-a}}.$$

Для неё применяется градиентный спуск с обратным распространением ошибки при функции потерь $E = (y - y_{\sim})^2$.

Ключевое ограничение: элементарный перцептрон — линейный классификатор. Он хорошо работает на линейно разделимых данных и ограничен на XOR, circles и spiral без нелинейного преобразования признаков.

3. Часть I. Генерирование обучающих и тестовых выборок

Во всех экспериментах точки генерируются на плоскости в диапазоне, близком к $[-6, 6]$. После добавления шума координаты ограничиваются этим интервалом, а затем выборка случайно перемешивается.

3.1. Общие параметры генерации

- Размер полной выборки: по умолчанию $N = 200$;
- Разбиение: 70% train, 30% test;
- Шум по признакам: $\varepsilon \sim \mathcal{N}(0, \sigma^2 I_2)$, по умолчанию используется `noise = 0.8`;
- Диапазон координат после генерации: $[-6, 6]$;
- `seed = 42`.

3.2. Обучающая, валидационная и тестовая выборки

В задаче классификации данные обычно делят на три части:

- **train** — используется для обновления параметров модели;
- **validation** — используется для подбора гиперпараметров;
- **test** — используется только для финальной оценки качества.

В текущей реализации используется разбиение **train/test** в пропорции $\frac{70}{30}$. На playground роль validation и test фактически играет отложенная (необучающая) часть выборки.

Переобучение проявляется как существенный разрыв ошибок: $L_{\text{train}} \ll L_{\text{test}}$. Обычно это связано с избыточной сложностью сети и малым объёмом обучающих данных.

3.3. Концентрические окружности (circles)

Математически классы формируются точками двух окружностей с радиусами $r_1 < r_2$:

$$x = (r \cos \theta, r \sin \theta), \quad \theta \sim U[0, 2\pi].$$

Класс 0 соответствует внутренней окружности, класс 1 — внешней. После генерации добавляется шум к обеим координатам.

```
import numpy as np

from config import AXIS_LIMIT, NOISE

def make_circles(n=200, noise=None, factor=0.45, seed=42):
    if noise is None:
        noise = NOISE
    rng = np.random.default_rng(seed)
    n1, n2 = n // 2, n - n // 2
    t1 = rng.uniform(0, 2 * np.pi, n1)
    t2 = rng.uniform(0, 2 * np.pi, n2)
    outer_radius = AXIS_LIMIT * 0.75
    inner_radius = outer_radius * factor
    inner = np.c_[inner_radius * np.cos(t1), inner_radius * np.sin(t1)]
```

```

outer = np.c_[outer_radius * np.cos(t2), outer_radius * np.sin(t2)]

x = np.vstack([inner, outer])
y = np.hstack([np.zeros(n1, dtype=int), np.ones(n2, dtype=int)])
x += rng.normal(0, noise, x.shape)
x = np.clip(x, -AXIS_LIMIT, AXIS_LIMIT)
idx = rng.permutation(n)
return x[idx], y[idx]

```

Задача нелинейно разделима: любая линейная граница пересекает оба кольца и не может корректно разделить классы.

Задание 2. Блок I — Circles (N = 1200)

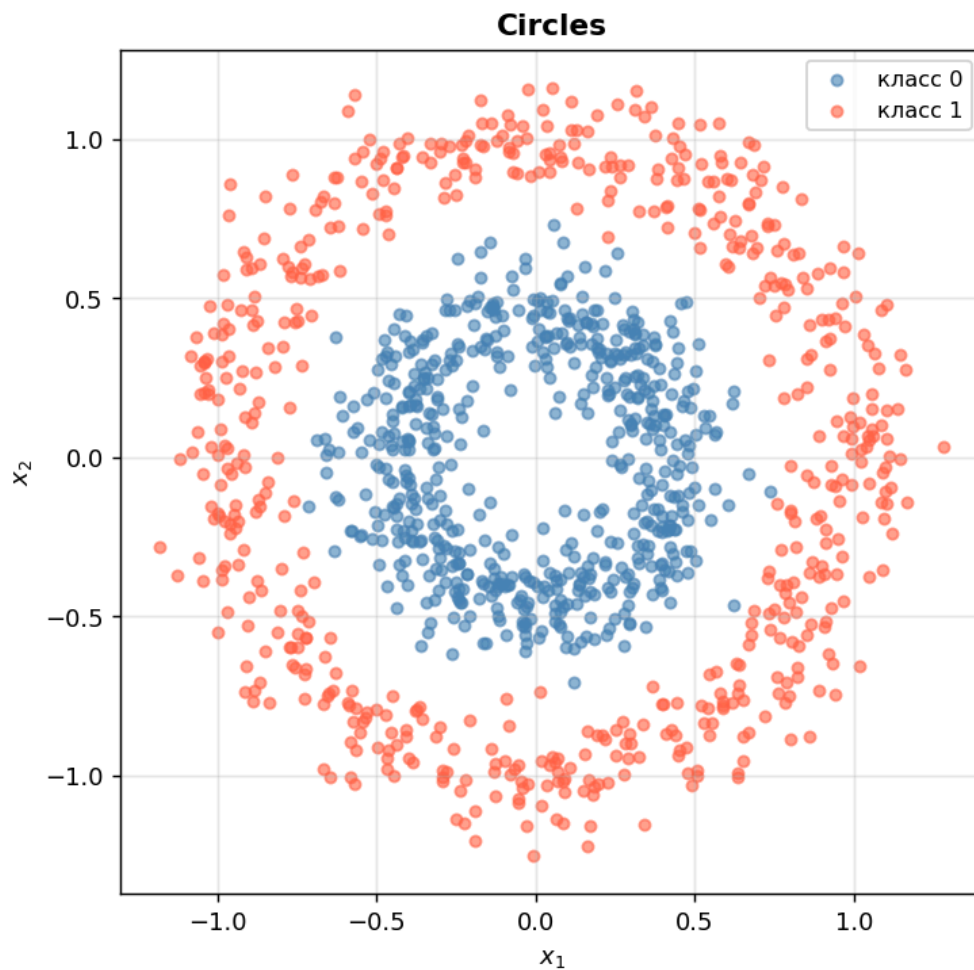


Рис. 1. Распределение circles: внутренняя окружность — класс 0, внешняя — класс 1, $N = 200$.

Переобучение в playground.tensorflow.org (пример настройки):

Наблюдается малый train-loss и заметно больший test-loss, а также избыточно «изломанная» граница решения.

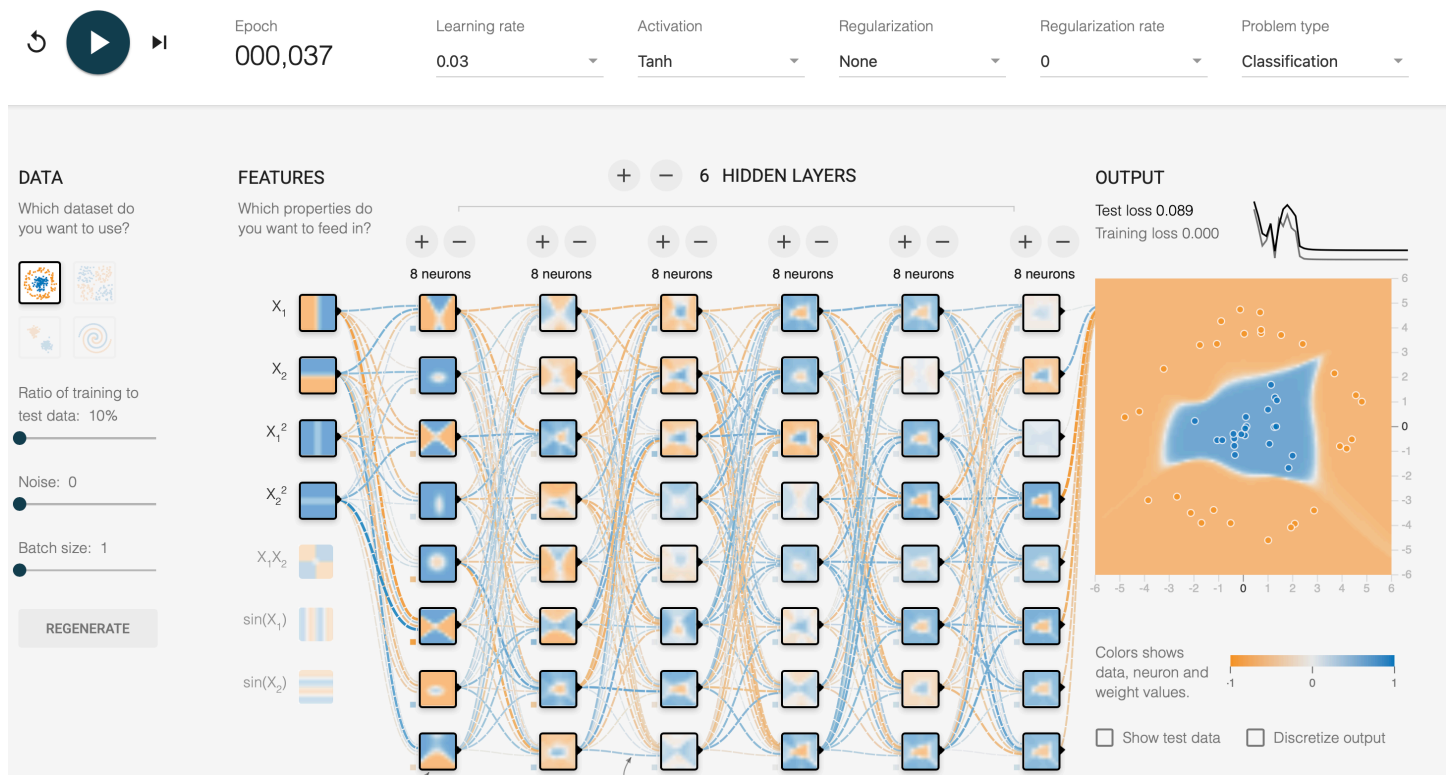


Рис. 2. Переобучение для circles в playground.tensorflow.org: training loss стремится к нулю, при этом test-loss остаётся заметно выше.

3.4. XOR-распределение

Формируются четыре кластера, расположенные около вершин квадрата $(-1, -1)$, $(-1, 1)$, $(1, -1)$, $(1, 1)$. Метка задаётся правилом XOR:

$$y = [x_1 x_2 < 0].$$

```
def make_xor(n=200, noise=None, seed=42):
    if noise is None:
        noise = NOISE
    rng = np.random.default_rng(seed)
    n4 = n // 4
    center = AXIS_LIMIT * 0.5
    centers = [(-center, -center), (-center, center), (center, -center), (center, center)]
    parts = [np.array(c) + rng.normal(0, noise, (n4, 2)) for c in centers]
    x = np.vstack(parts)
    x = np.clip(x, -AXIS_LIMIT, AXIS_LIMIT)
    y = ((x[:, 0] * x[:, 1]) < 0).astype(int)
    idx = rng.permutation(len(x))
    return x[idx], y[idx]
```

Задача нелинейно разделима: один линейный классификатор не может выделить диагонально противоположные области как один класс.

Задание 2. Блок I — XOR (N = 1200)

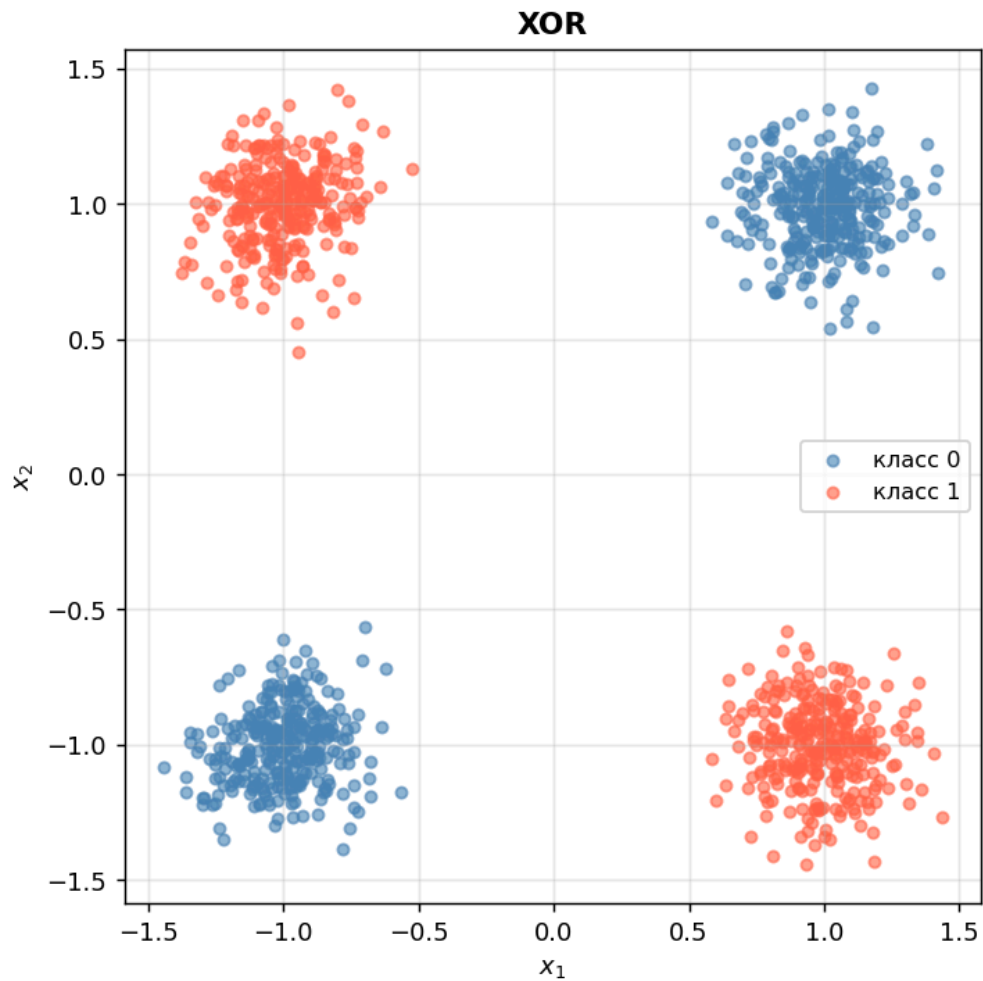


Рис. 3. Распределение XOR: четыре кластера, метки по правилу $x_1 x_2 < 0$, $N = 200$.

Переобучение (пример настройки playground):

Получается практически идеальная подгонка train и заметный провал на test при усложнении границы.

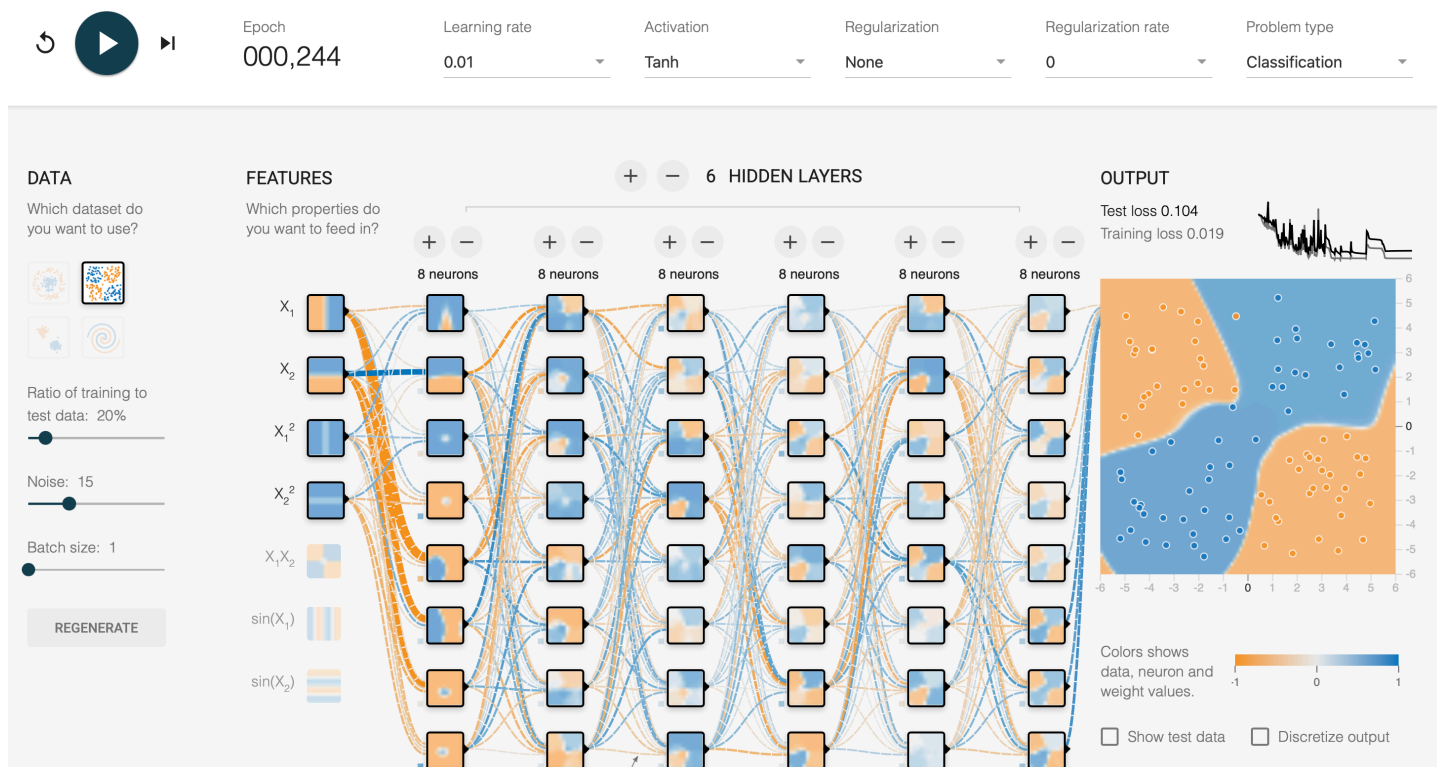


Рис. 4. Переобучение для XOR в playground.tensorflow.org: сеть формирует сложную нелинейную область решения и подгоняет обучающую выборку почти без ошибки.

3.5. Гауссовские кластеры (blobs)

Каждый класс моделируется двумерным нормальным распределением:

$$x \mid y = c \sim \mathcal{N}(\mu_c, \sigma_c^2 I_2).$$

```
def make_blobs(n=200, noise=None, seed=42):
    if noise is None:
        noise = NOISE
    rng = np.random.default_rng(seed)
    n1, n2 = n // 2, n - n // 2
    center = AXIS_LIMIT * 0.5
    c0 = rng.normal(0, noise, (n1, 2)) + np.array([-center, -center])
    c1 = rng.normal(0, noise, (n2, 2)) + np.array([center, center])

    x = np.vstack([c0, c1])
    x = np.clip(x, -AXIS_LIMIT, AXIS_LIMIT)
    y = np.hstack([np.zeros(n1, dtype=int), np.ones(n2, dtype=int)])
    idx = rng.permutation(n)
    return x[idx], y[idx]
```

При малом шуме и достаточном расстоянии между центрами задача почти линейно разделима; при росте `noise` появляется перекрытие классов и ошибка классификации возрастает.

Задание 2. Блок I — Blobs (N = 1200)

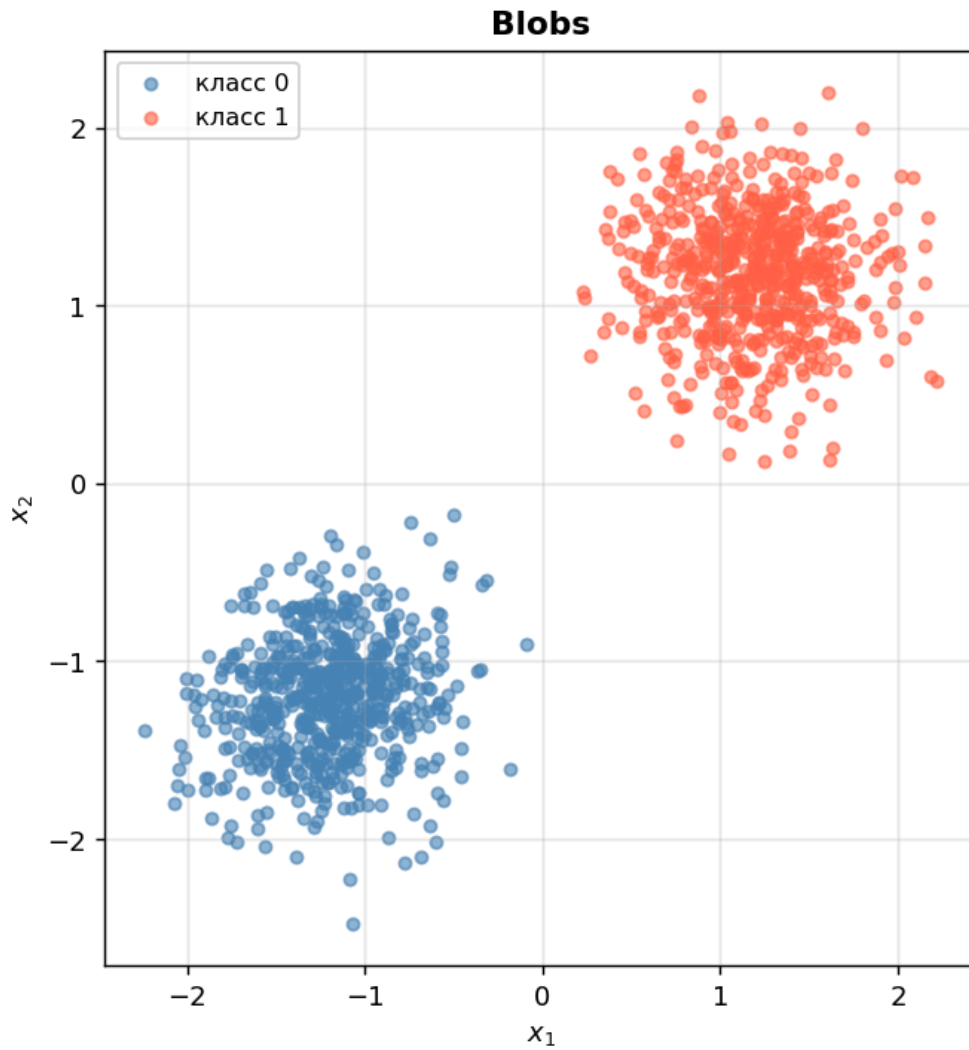


Рис. 5. Распределение blobs: два гауссовских кластера, $N = 200$.

Переобучение (пример настройки playground):

Даже на сравнительно простой структуре при очень малом train наборе модель начинает запоминать частные флуктуации точек.

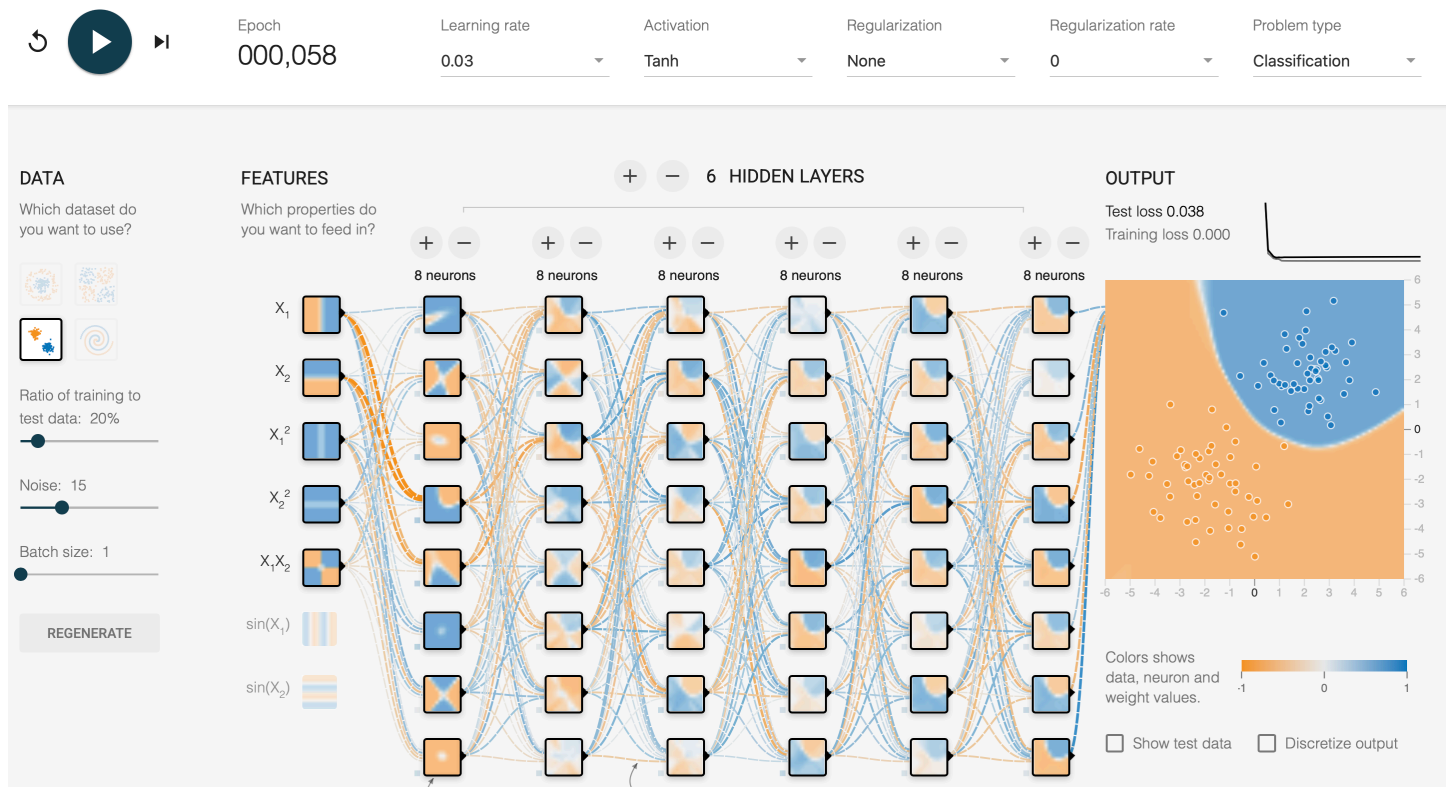


Рис. 6. Переобучение для blobs в playground.tensorflow.org: граница решения начинает описывать частные особенности подвыборки вместо общей линейной структуры.

3.6. Спирали Архимеда (spiral)

Классические двухклассовые спирали задаются в полярных координатах:

$$r = a + b\theta, \quad x_1 = r \cos \theta, \quad x_2 = r \sin \theta.$$

Вторая спираль получается сдвигом фазы на π .

```
def make_spiral(n=200, noise=None, turns=2.5, seed=42):
    if noise is None:
        noise = NOISE
    rng = np.random.default_rng(seed)
    n1, n2 = n // 2, n - n // 2
    t1 = np.linspace(0, turns * np.pi, n1)
    t2 = np.linspace(0, turns * np.pi, n2)
    max_radius = AXIS_LIMIT * 0.75
    r1 = max_radius * t1 / (turns * np.pi)
    r2 = max_radius * t2 / (turns * np.pi)

    x1 = np.c_[r1 * np.cos(t1), r1 * np.sin(t1)]
    x2 = np.c_[r2 * np.cos(t2 + np.pi), r2 * np.sin(t2 + np.pi)]

    x = np.vstack([x1, x2])
    x += rng.normal(0, noise, x.shape)
    x = np.clip(x, -AXIS_LIMIT, AXIS_LIMIT)
    y = np.hstack([np.zeros(n1, dtype=int), np.ones(n2, dtype=int)])
    idx = rng.permutation(n)
    return x[idx], y[idx]
```

Задача принципиально нелинейно разделима и требует гибкой нелинейной границы.

Задание 2. Блок I — Spiral (N = 1200)

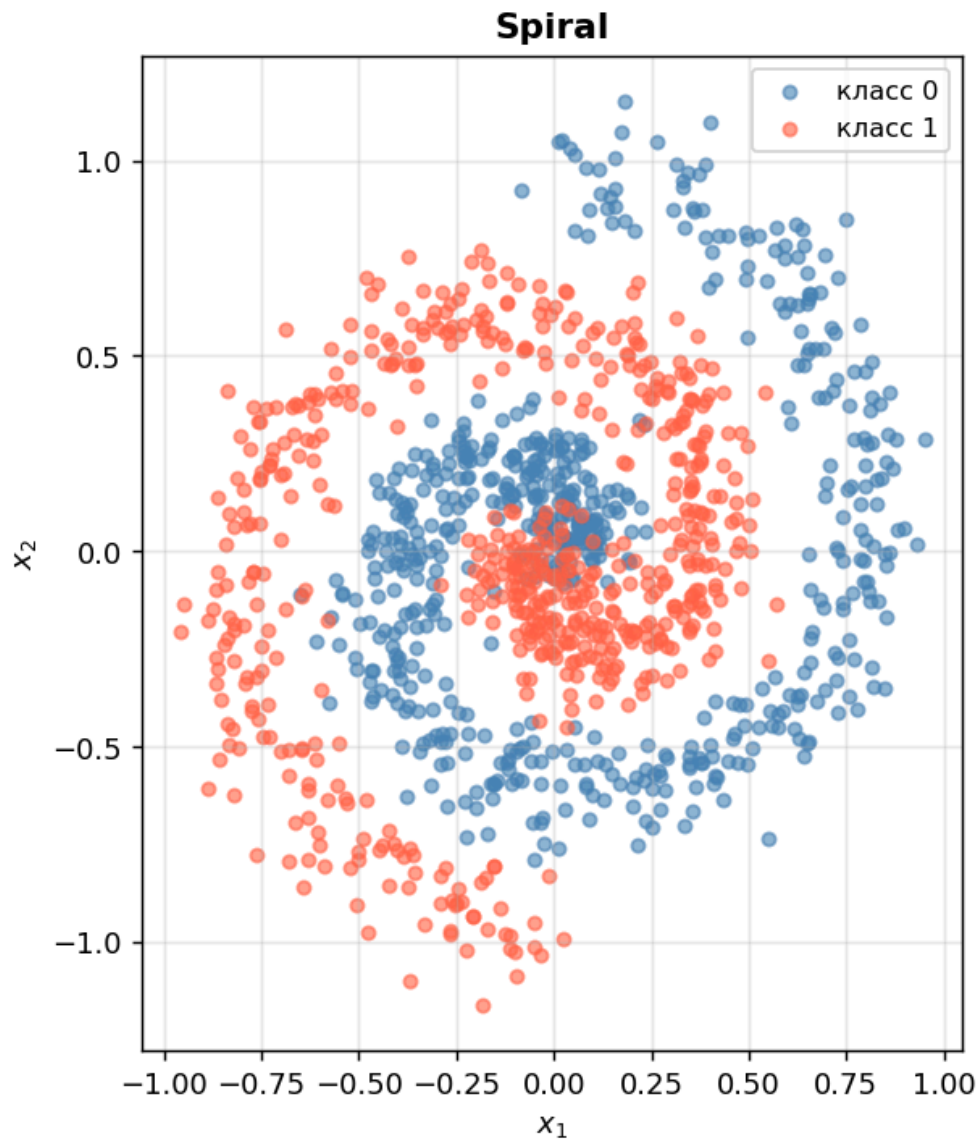


Рис. 7. Распределение spiral: две спирали Архимеда, сдвинутые на π , $N = 200$.

Переобучение (пример настройки playground):

Наблюдается сложная «рваная» граница с низким train-loss и заметно более высоким test-loss.

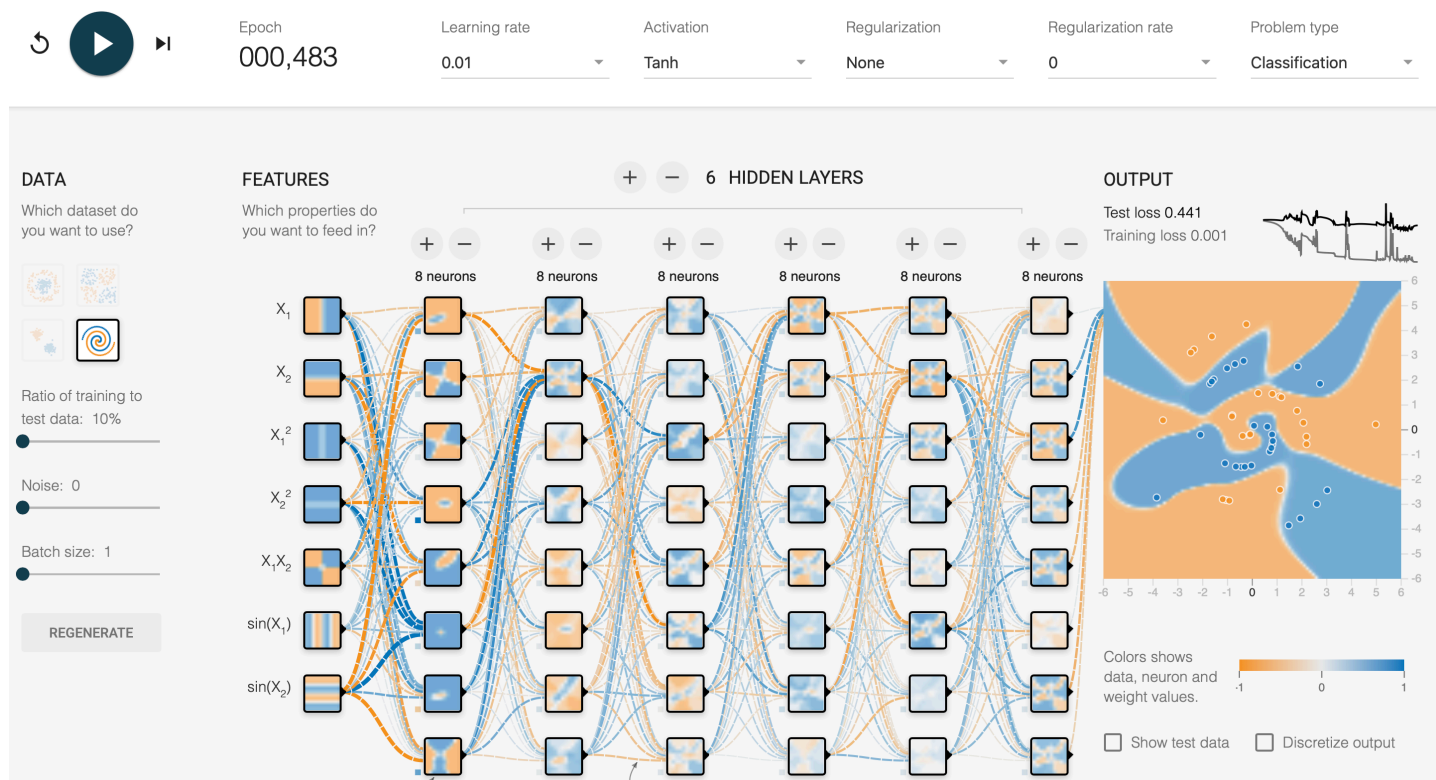


Рис. 8. Переобучение для spiral в playground.tensorflow.org: модель строит сложную извилистую границу и демонстрирует существенный разрыв между train и test-loss.

4. Часть II. Реализация элементарного перцептрона

4.1. Математическая модель

Линейная комбинация признаков:

$$a = w_0 + w_1 x_1 + w_2 x_2 = w_{\sim}^T x_{\sim}.$$

Выход модели:

$$y_{\sim} = \varphi(a).$$

Для ступенчатой активации решение принимается по знаку a . Для сигмоиды $y_{\sim} \in (0, 1)$, а класс определяется порогом 0.5.

4.2. Код реализации

```
import numpy as np
import time

class Perceptron:
    def __init__(self, activation='step', lr=0.1, n_epochs=300, seed=42):
        self.activation = activation
        self.lr = lr
        self.n_epochs = n_epochs
```

```

self.seed = seed
self.w = None
self.train_time = 0.0

def _sigmoid(self, a):
    return 1.0 / (1.0 + np.exp(-np.clip(a, -500, 500)))

def fit(self, x, y):
    rng = np.random.default_rng(self.seed)
    xb = np.c_[np.ones(len(x)), x]
    self.w = rng.normal(0, 0.01, xb.shape[1])
    y_norm = 2 * y - 1
    t0 = time.perf_counter()

    if self.activation == 'step':
        for _ in range(self.n_epochs):
            changed = False
            for i in range(len(xb)):
                pred = 1 if (xb[i] @ self.w) >= 0 else 0
                if pred != y[i]:
                    self.w += self.lr * y_norm[i] * xb[i]
                    changed = True
            if not changed:
                break
    else:
        for _ in range(self.n_epochs):
            yh = self._sigmoid(xb @ self.w)
            delta = -2.0 * (y - yh) * yh * (1.0 - yh)
            grad = (delta @ xb) / len(xb)
            self.w -= self.lr * grad

    self.train_time = time.perf_counter() - t0

def predict(self, x):
    xb = np.c_[np.ones(len(x)), x]
    a = xb @ self.w
    if self.activation == 'step':
        return (a >= 0).astype(int)
    return (self._sigmoid(a) >= 0.5).astype(int)

```

4.3. Где это реализовано в проекте

- Файл `perceptron.py`, метод `fit`:
 - ветка `if self.activation == 'step'` — обучение по правилу сходимости перцептрона (обновление только при ошибке);
 - ветка `else` — градиентный спуск для сигмоидальной модели по производной MSE.
- Файл `perceptron.py`, метод `predict` — пороговая классификация (`a >= 0` для `step` и `sigma(a) >= 0.5` для `sigmoid`).

- Файл `block2_main.py`, константа `ACTIVATIONS` — реальные гиперпараметры запуска читаются из переменных окружения `SMGM02_STEP_LR`, `SMGM02_STEP_EPOCHS`, `SMGM02_SIGMOID_LR`, `SMGM02_SIGMOID_EPOCHS` (по умолчанию: `lr = 0.03`, `epochs = 1000` для обеих моделей).
- Файл `block2_main.py`, функция `train_all` — полный цикл генерация $\rightarrow$ `split_data` $\rightarrow$ `fit` $\rightarrow$ `predict` $\rightarrow$ `confusion_matrix`.

4.4. Вычислительный граф и локальные производные

Для сигмоидального перцептрона вычислительный граф показан на рисунке ниже.

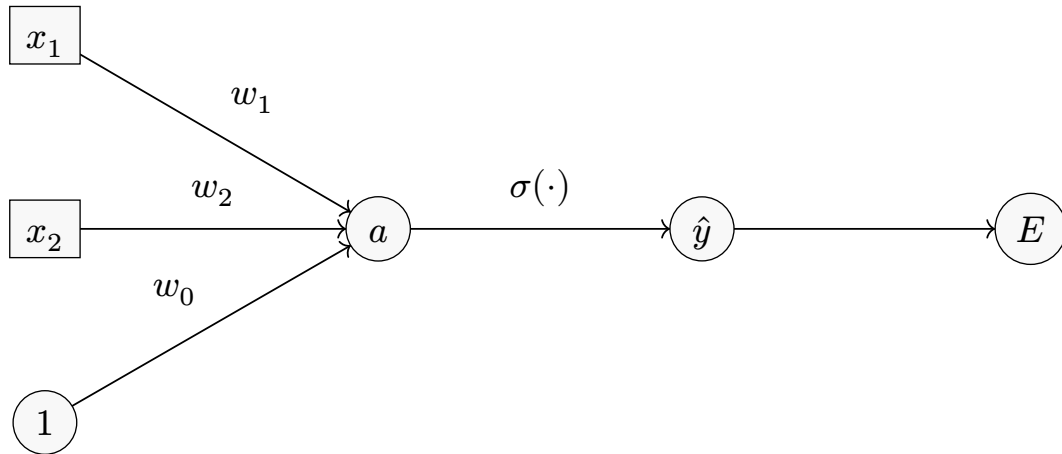


Рис. 9. Вычислительный граф элементарного перцептрона.

Граф отражает последовательность вычислений при прямом и обратном проходе:

1. **Входной слой.** Три узла слева — признаки x_1 , x_2 и фиктивный вход 1 (bias). Каждый из них соединён с узлом a ребром с соответствующим весом w_0, w_1, w_2 .

2. **Линейная комбинация.** В узле a вычисляется взвешенная сумма:

$$a = w_0 \cdot 1 + w_1 x_1 + w_2 x_2 = \tilde{w}^T \tilde{x}.$$

3. **Функция активации.** Ребро $\sigma(\cdot)$ применяет сигмоиду:

$$\hat{y} = \sigma(a) = \frac{1}{1 + e^{-a}}.$$

Выход $\hat{y} \in (0, 1)$ интерпретируется как вероятность класса 1.

4. **Функция потерь.** В узле E вычисляется квадратичная ошибка:

$$E = (y - \hat{y})^2.$$

При **обратном проходе** ошибка E распространяется по тем же рёбрам справа налево. По правилу цепочки на каждом ребре перемножаются локальные производные — это и есть алгоритм `backpropagation`.

Локальные производные:

$$\frac{\partial E}{\partial y^-} = -2(y - y^-),$$

$$\frac{\partial y^-}{\partial a} = y_{1-y^-},$$

$$\frac{\partial a}{\partial w_i} = x_i.$$

По правилу цепочки:

$$\frac{\partial E}{\partial w_i} = \frac{\partial E}{\partial y^-} \cdot \frac{\partial y^-}{\partial a} \cdot \frac{\partial a}{\partial w_i} = -2(y - y^-)y_{1-y^-}x_i.$$

Обновление весов:

$$w_i \leftarrow w_i - \eta \frac{\partial E}{\partial w_i}.$$

Для ступенчатой функции производная не определена в нуле и равна нулю почти всюду, поэтому используется не backprop, а правило коррекции ошибок перцептрона.

4.5. Обучение и оценка

Разбиение выборок на train/test выполняется в пропорции $\frac{70}{30}$:

```
def split_data(x, y, test_ratio=0.3, seed=42):
    rng = np.random.default_rng(seed)
    idx = rng.permutation(len(x))
    cut = int(len(x) * (1 - test_ratio))
    tr, te = idx[:cut], idx[cut:]
    return x[tr], y[tr], x[te], y[te]
```

Параметры обучения:

- параметры step: learning_rate = 0.03, n_epochs = 1000;
- параметры sigmoid: learning_rate = 0.03, n_epochs = 1000.

Параметры можно менять отдельно через GUI (поля для step и sigmoid).

		Predicted Class		
		Positive	Negative	
Actual Class	Positive	True Positive (TP)	False Negative (FN) Type II Error	Sensitivity $\frac{TP}{(TP + FN)}$
	Negative	False Positive (FP) Type I Error	True Negative (TN)	Specificity $\frac{TN}{(TN + FP)}$
		Precision $\frac{TP}{(TP + FP)}$	Negative Predictive Value $\frac{TN}{(TN + FN)}$	Accuracy $\frac{TP + TN}{(TP + TN + FP + FN)}$

Рис. 10. Как интерпретировать confusion matrix для модели машинного обучения

4.5.1. Визуализация результатов блока 2

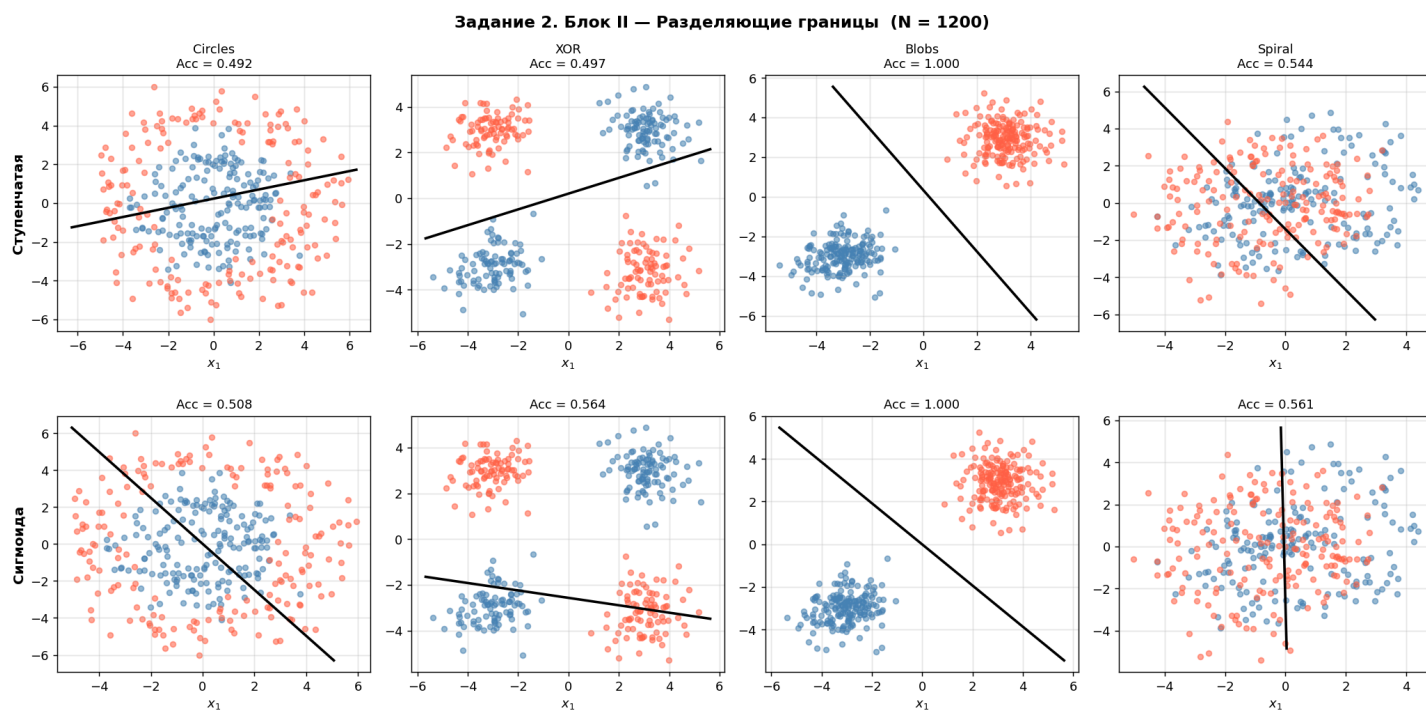


Рис. 11. Разделяющие границы перцептрона для всех распределений (step и sigmoid).

Задание 2. Блок II — Матрицы ошибок (N = 1200)

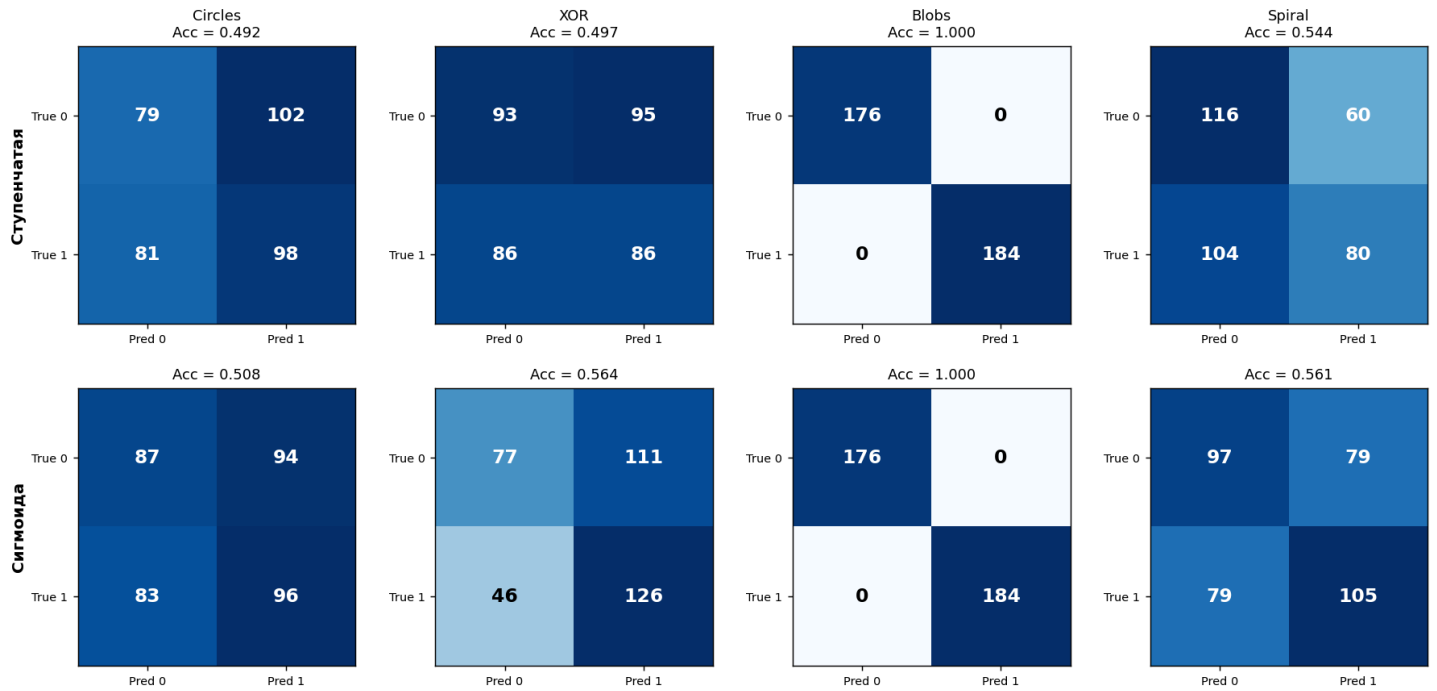


Рис. 12. Матрицы ошибок перцептрона для всех распределений (step и sigmoid).

4.5.2. Матрицы ошибок (confusion matrix)

Модель	Распределение	TN	FP	FN	TP
Step	Circles	79	102	81	98
Step	XOR	93	95	86	86
Step	Blobs	176	0	0	184
Step	Spiral	116	60	104	80
Sigmoid	Circles	87	94	83	96
Sigmoid	XOR	77	111	46	126
Sigmoid	Blobs	176	0	0	184
Sigmoid	Spiral	97	79	79	105

Таблица 1. Матрицы ошибок (сводные значения по test-выборке).

4.5.3. Классификационные метрики

Модель	Распределение	Accuracy	Precision	Recall	F1
Step	Circles	0.492	0.490	0.548	0.517
Step	XOR	0.497	0.475	0.500	0.487
Step	Blobs	1.000	1.000	1.000	1.000
Step	Spiral	0.544	0.571	0.435	0.494
Sigmoid	Circles	0.508	0.505	0.536	0.520
Sigmoid	XOR	0.564	0.532	0.733	0.617
Sigmoid	Blobs	1.000	1.000	1.000	1.000
Sigmoid	Spiral	0.561	0.571	0.571	0.571

Таблица 2. Сравнение качества классификации на test-выборке.

4.6. Сравнение двух моделей

Распределение	Step: время, с	Sigmoid: время, с	Вывод
Circles	1.070	0.012	Сигмоида точнее, а Step медленнее из-за не-сходимости
XOR	1.087	0.012	Обе модели ограничены линейностью
Blobs	0.001	0.012	Обе модели эффективны; Step сходится быстро
Spiral	1.061	0.012	Качество ограничено, нужен MLP

Таблица 3. Сравнение времени обучения и итогового качества.

Итог сравнения:

- Ступенчатый перцептрон на нелинейных данных не сходится и проходит все эпохи (+-1с), тогда как сигмоида обучается стабильно быстро (+-0.012с) за счёт непрерывного градиента.
- На линейно разделимых данных (Blobs) Step сходится досрочно за 0.001с и даёт идеальный результат.
- На XOR, Circles и Spiral обе версии ограничены линейной границей — точность близка к случайному угадыванию (+-0.5).

5. Выводы

В первой части реализованы генераторы четырёх типов двумерных выборок, включая моделирование шумовой ошибки по признакам. Показано, что тип геометрии распределения непосредственно определяет сложность задачи классификации и требуемую мощность модели.

На playground.tensorflow.org продемонстрировано переобучение: при большой глубине сети, отсутствии регуляризации и малой доле обучающих данных достигается низкая ошибка на train и более высокая на test.

Во второй части реализован элементарный перцептрон в двух вариантах: со ступенчатой активацией (правило сходимости) и сигмоидальной активацией (градиентный спуск с backprop). Полученные confusion matrix и метрики подтверждают: перцептрон эффективен на линейно разделимых данных (blobs) и принципиально ограничен на нелинейных распределениях.

Главный вывод: обобщающая способность определяется балансом между сложностью данных и сложностью модели. Для сложных нелинейных структур необходимы либо нелинейные признаки, либо многослойные нейросети.

6. Структура проекта

Проект организован по модульному принципу для удобства разработки и поддержки:

```
SMGMO/
├── lab2/
│   ├── main.typ          # отчёт (Typst)
│   ├── config.py         # общие параметры экспериментов
│   ├── generators.py      # генераторы circles, xor, blobs, spiral
│   ├── perceptron.py     # элементарный перцептрон и метрики
│   ├── block1_main.py    # визуализация выборок
│   ├── block2_main.py    # обучение и сравнение перцептрона
│   ├── gui_launcher.py   # графический запуск блоков
│   ├── task2_block1_circles.png # выборка circles
│   ├── task2_block1_xor.png   # выборка xor
│   ├── task2_block1_blobs.png  # выборка blobs
│   ├── task2_block1_spiral.png  # выборка spiral
│   ├── task2_block2_boundaries.png # границы решений (блок 2)
│   ├── task2_block2_cm.png     # матрицы ошибок (блок 2)
│   ├── matrix.png             # схема интерпретации confusion matrix
│   ├── overfitting_circles.png # скриншот переобучения circles
│   ├── overfitting_xor.png     # скриншот переобучения xor
│   ├── overfitting_blobs.png   # скриншот переобучения blobs
│   └── overfitting_spiral.png  # скриншот переобучения spiral
└── ...
```

7. Репозиторий

Исходный код проекта доступен на GitHub: <https://github.com/Q1zin/SMGMO.git>